

# CS6370: Natural Language Processing

## Assignment 1

Release Date: 27th Feb 2024

Deadline: 11th March 2024

Name:

Roll No.:

|                      |          |
|----------------------|----------|
| Siddhartha Guggulotu | ME20B167 |
|----------------------|----------|

### General Instructions:

1. The template for the code (in Python) is provided in a separate zip file. You are expected to fill in the template wherever instructed. Note that any Python library, such as nltk, stanfordcorenlp, spacy, etc, can be used.
2. The programming questions for the Spell Check and WordNet parts need to be done in separate Python files.
3. A folder named 'Roll\_number.zip' that contains a zip of the code folder and your responses to the questions (a PDF of this document with the solutions written in the text boxes) must be uploaded on Moodle by the deadline.
4. Any submissions made after the deadline will not be graded.
5. Answer the theoretical questions concisely. All the codes should contain proper comments.
6. The institute's academic code of conduct will be strictly enforced.

---

The goal of this assignment is to build a search engine from scratch, which is an example of an Information Retrieval system. In the class, we have seen the various modules that serve as the building blocks for a search engine. We will be progressively building the same as the course progresses. This assignment requires you to build a basic text processing module that implements sentence segmentation, tokenization, stemming/lemmatization, spell check, and stopword removal. You will also explore some aspects of WordNet as a part of this assignment. The Cranfield dataset, which has been uploaded, will be used for this purpose.

Part 1: Sentence Segmentation

[Theory + Implementation]

1. Suggest a simplistic top-down approach to sentence segmentation for English texts. Do you foresee issues with your proposed approach in specific situations? Provide supporting examples and possible strategies that can be adopted to handle these issues. [2 marks]

A simplistic top-down approach for sentence segmentation would be identifying punctuation marks like periods, question marks, and exclamation marks which most commonly occur at the end of a sentence to split the text into sentences.

Issues:

1. Abbreviations: Dr., Mr. and other such designations contain a period.
2. Sentences like Hi! Good to see you. Would be identified as two different sentences.
3. Question tags: It is good. Isn't it? will also be identified as two different sentences.

Strategies to handle:

1. Custom to rules to handle specific cases like Dr. , Mr.
2. Checking the number of words that occur before a punctuation mark, to identify issues like question tags.
3. Contextual analysis to determine if the punctuation indicates a sentence boundary.

2. Python NLTK is one of the most commonly used packages for Natural Language Processing. What does the Punkt Sentence Tokenizer in NLTK do differently from the simple top-down approach? [1 marks]

The Punkt Sentence Tokenizer uses pre-trained statistical models to identify sentence boundaries based on features like punctuation, capitalization and context. It also considers abbreviations, ellipses and other linguistic nuances.

3. Perform sentence segmentation on the documents in the Cranfield dataset using:
  - a. The proposed top-down method and

- b. The pre-trained Punkt Tokenizer for English
- State a possible scenario where
- Your approach performs better than the Punkt Tokenizer
  - Your approach performs worse than the Punkt Tokenizer
- [4 marks]

```
class SentenceSegmentation():
    def naive(self, text):

        segmentedText = []
        current_sentence = ""

        for char in text:
            current_sentence += char #adding a character to the sentence until a ., !, ? appears
            if char in ['.', '!', '?']:
                segmentedText.append(current_sentence.strip()) #adding the sentence to the list
                current_sentence = "" #reinitializing sentence to empty string

        # Add the last sentence if it's not empty
        if current_sentence.strip():
            segmentedText.append(current_sentence.strip())

        return segmentedText
```

- a. Proposed top-down approach

```
class SentenceSegmentation():
    def punkt(self, text):

        #try - except for easy debugging in case of errors
        try:
            segmented_text = sent_tokenize(text) #using punkt for sentence segmentation
        except Exception as e:
            raise Exception("Error occurred during sentence segmentation:", e)

        return segmented_text
```

- b. Pre-trained punkt tokenizer for English

1. Scenario my approach > Punkt

- When the text is domain specific on which punkt wasn't trained but the naive approach could have additional rules specific to that text.
- Example:
  - Bulleted points listing out steps to cook a meal.
  - Each sentence might have a lot of separators but the naive approach, could just segment sentences based on the presence of a newline character.

2. Scenario Punkt > my approach

- When the context is general and the text contains lot of complex sentences, abbreviations.
- Example:
  - Abbreviations like i.e., U.S.A, numbers like 2.4 were handled by punkt whereas my approach recognized the

e as a separate word.

## Part 2: Tokenization

[Theory + Implementation]

1. Suggest a simplistic top-down approach for tokenization in English text. Identify specific situations where your proposed approach may fail to produce expected results. [2 marks]

A simplistic top-down tokenisation approach would use delimiters like spaces, tabs and new lines to identify word boundaries.

Failure cases:

1. Word contractions like wouldn't or don't contain internal apostrophes, which can lead to incorrect tokenization. This should be tokenized as would , n't and do , n't.
2. It fails in tokenizing words separated by punctuations without a space between them.

2. Study about NLTK's Penn Treebank tokenizer. What type of knowledge does it use - Top-down or Bottom-up? [1 mark]

The Penn Treebank Tokenizer uses a **bottom-up** approach. It handles contractions, hyphenated words and punctuations more accurately.

3. Perform word tokenization of the sentence-segmented documents using
  - a. The proposed top-down method and
  - b. Penn Treebank Tokenizer

State a possible scenario along with an example where:

- a. Your approach performs better than Penn Treebank Tokenizer
- b. Your approach performs worse than Penn Treebank Tokenizer

[4 marks]

```

class Tokenization():
    def naive(self, text):

        tokenizedText = []

        for sentence in text:
            # Split each sentence into tokens using spaces, tabs and newlines
            tokens = re.split(r'\s+/\t+/\n+', sentence)
            tokenizedText.append(tokens)

        return tokenizedText

```

a. The proposed Top-down method

```

class Tokenization():
    def pennTreeBank(self, text):

        tokenizedText = []

        for sentence in text:
            #try - except for easy debugging
            try:
                tokens = word_tokenize(sentence) #using NLTK's word tokenize function
            except Exception as e:
                raise Exception("Error occurred during tokenization:", e)
            tokenizedText.append(tokens)

        return tokenizedText

```

b. Penn Treebank Tokenizer

1. Scenario where My approach > PTB:

a. Formal text with minimal contractions and hyphenated words, domain specific words that we can make our own rules for.

b. Example:

- i. My o/p “DNA Sequence” (medical documents where we need this to be tokenized without space separation can be modified by adding an extra rule)
- ii. Ptb o/p “DNA”, “Sequence”

2. Scenario where Penn Treebank Tokenizer > My approach:

a. When there are too many special characters, like in a novel.

b. Example:

- i. My output "(the", "?slip?", "effect)"
- ii. Ptb output "(", "the", "?", "slip", "?", "effect", ")"

1. What is the difference between stemming and lemmatization? Give an example to illustrate your point. [1 marks]

Stemming: Reducing words to their root form by removing suffixes, it may not always provide valid words.

Ex: running -> run , dancing -> danc

Lemmatization: Reducing words to their root form by considering the context and part of speech to ensure valid words.

Ex: better -> good, went -> go

2. Using Porter's stemmer, perform stemming/lemmatization on the word-tokenized text from the Cranfield Dataset. [1 marks]

```
class InflectionReduction:
    def reduce(self, text):
        reducedText = []
        stemmer = PorterStemmer() #initialize porter stemmer object
        for sentence in text:
            #iteration over all tokens and stemming
            stemmed_sentence = [stemmer.stem(token) for token in sentence]
            reducedText.append(stemmed_sentence)
        return reducedText
```

- a. Function to lemmatize/stem the tokens using Porter's Stemmer

## Part 4: Stopword Removal

[Theory + Implementation]

1. Remove stopwords from the tokenized documents using a curated list, such as the list of stopwords from NLTK. [1 marks]

```
class StopwordRemoval():  
    def fromList(self, text):  
  
        stopwordsRemovedText = []  
  
        #get the set of nltk english stopwords  
        stop_words = set(stopwords.words("english"))  
  
        for sentence in text:  
            #adding the words that are not in stop_words to filtered_sentence  
            filtered_sentence = [word for word in sentence if word.lower() not in stop_words]  
            stopwordsRemovedText.append(filtered_sentence)  
  
        return stopwordsRemovedText
```

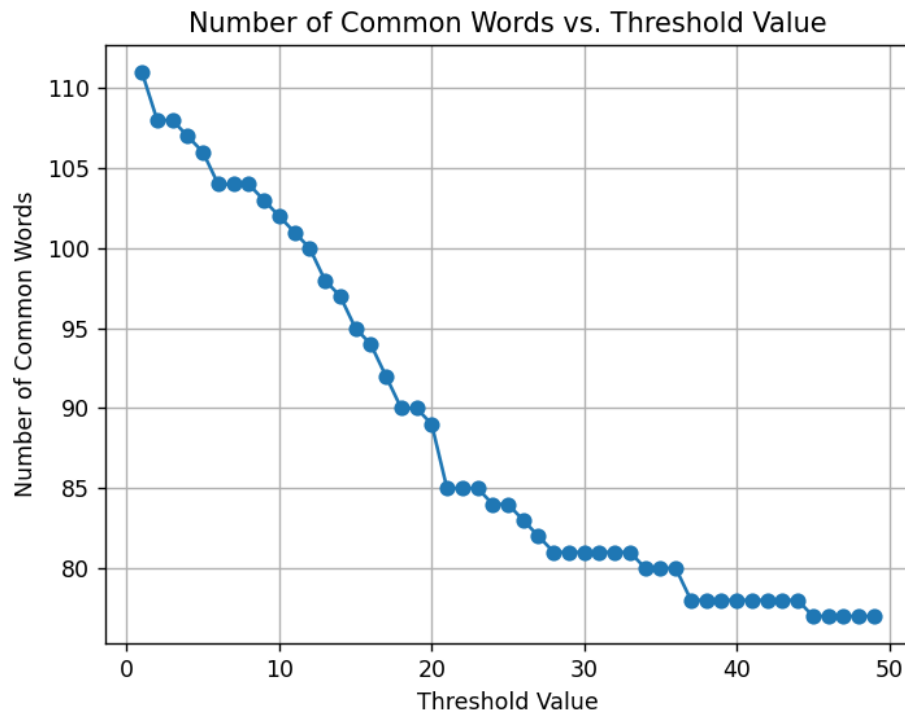
a. Function to create a new list with stop words removed

2. Can you suggest a bottom-up approach for creating a list of stopwords specific to the corpus of documents? [1 marks]

Bottom-up approach: Using metrics like term frequency and inverse document frequency, the terms with high frequency have the highest probability of being stop words. Manual review is required to check if some useful words that occur frequently are not removed.

3. Implement the strategy proposed in the previous question and compare the stopwords obtained with those obtained from NLTK on the Cranfield dataset. [2 marks]

The below graph can be obtained by running the command "Python stopwordsIdentify.py" make sure to change the path to dataset in the same file.



The plot represents the number of common words in the list of stopwords made using the term frequency and the nltk stopwords list against the threshold value above which the term is considered a stop word.

1. The plot shows that the slope is almost zero starting from 18 after a huge drop. This suggests that 18 is an ideal threshold value, where most of the non stopwords have been removed from the list.
2. The next best threshold value is 21 and then 28, If a loss of non stopwords is observed after stop word removal, we can try the next threshold values where the slope is going to zero.
3. Beyond 21 the slope reduction is minimal which shows that increasing threshold is not filtering out many words, which suggests that most of them are stop words.

From the graph 18, 21, 28 are ideal threshold values based on requirement.

1. Given a set of queries  $Q$  and a corpus of documents  $D$ , what would be the number of computations involved in estimating the similarity of each query with every document? Assume you have access to the TF-IDF vectors of the queries and documents over the vocabulary  $V$ . [1 marks]

As we have access to the TF-IDF vectors, we have to do a cosine similarity operation between each query and all the documents.

Assuming number of queries =  $n$

Number of documents =  $m$

For  $q$  in queries: ( $n$  operations)

Magnitude of query  $\sqrt{\text{matrix mul } W[q] W[q]}$

For  $d$  in docs: ( $m$  operations)

Matrix multiplication  $W[q] \cdot W[d]$

Magnitude of doc  $\sqrt{\text{matrix mul } W[d] W[d]}$

Divide to get  $\cos \theta$

This would require  $n \times m$  steps.

2. Suggest how the idea of the inverted index can help reduce the time complexity of the approach in (1). You can introduce additional variables as needed. [3 marks]

Inverted index uses a dictionary or similar data structure that maps the words to documents in which they occur.

To build an inverted index we still have to go through the  $n \times m$  steps, but it only happens once.

With an inverted index, we can directly access the documents related to the words in the query in  $O(1)$  time. The only comparison is between the lists corresponding to the words in the query so as to return a document only once and check for duplicates.

This comparison takes linear time  $O(n)$  where  $n$  is the length of the largest

list of docs.

1. Construct a vocabulary V of all the types (unique tokens) from the Cranfield dataset. You may additionally filter out alpha-numeric types. Represent each type in V as a vector in a vector space spanned by all possible bigrams of the English alphabet ('aa,' 'ab,' 'ac,'... 'zz'). Given the typos - 'boundery', 'transiant', 'aerplain' - find the top 5 candidate corrections corresponding to each. [5 marks]

\*\*\*

**Note: To run this code use the command**

```
> "Python main.py -custom -dataset "path to dataset" -out_folder  
"path to output" -segmenter punkt -tokenizer ptb"  
> Enter custom query  
> Boundery Transiant Aerplain
```

**The output will be written to a file named "spell\_checked\_queries.txt"**

\*\*\*

O/P:

1. Boundery:

a. [[["bound", 0.7559289460184544], ["boundari",  
0.7142857142857142], ["unbound", 0.6681531047810609],  
["bounari", 0.6172133998483676], ["underli",  
0.6172133998483676]],

2. Transiant:

a. [["transient", 0.7826237921249264], ["transit",  
0.7745966692414834], ["transon", 0.6454972243679029],  
["radiant", 0.6454972243679029], ["entranc",  
0.6454972243679029]],

3. Aerplain:

a. [["plain", 0.7559289460184544], ["explain",  
0.6172133998483676], ["airplan", 0.6172133998483676],  
["aeroplan", 0.5714285714285714], ["unexplain",

0.5345224838248487]]]]

```
def buildVocabulary(docs_list):  
    tokens = []  
  
    # Extract all alphanumeric tokens from the processed documents  
    for docs in docs_list:  
        for doc in docs:  
            for token in doc:  
                if (token.isalnum() and isinstance(token, str)):  
                    tokens.append(token)  
  
    # Find unique tokens  
    unique_tokens = set(tokens)  
  
    # Initialize an empty dictionary to store vocabulary vectors  
    vocabulary_vectors = {}  
  
    # Loop through each unique token to generate its vector representation  
    for token in unique_tokens:  
        # Generate vector representation for the token  
        vector = word_to_vector(token)  
  
        # Store the token and its vector representation in the vocabulary dictionary  
        vocabulary_vectors[token] = vector  
  
    # Return the vocabulary dictionary  
    return vocabulary_vectors
```

a. Function to build vocabulary vectors

```
def word_to_vector(word):  
    alphabet = "abcdefghijklmnopqrstuvwxyz"  
    #creating all possible bigrams from the english alphabet  
    bigrams = [a + b for a, b in product(alphabet, repeat=2)]  
  
    vector = [0] * len(bigrams)  
    for i in range(len(word) - 1):  
        #picking consecutive letters to check if present in bigram  
        bigram = word[i:i+2]  
        if bigram in bigrams:  
            #if the bigram is present update vector position to 1 else remains as 0  
            vector[bigrams.index(bigram)] += 1  
  
    return vector
```

b. Function that converts word to vector using bigrams

```
def cosine_similarity(vec1, vec2):  
    dot_product = sum(a * b for a, b in zip(vec1, vec2))  
    magnitude1 = math.sqrt(sum(a**2 for a in vec1))  
    magnitude2 = math.sqrt(sum(b**2 for b in vec2))  
    if magnitude1 == 0 or magnitude2 == 0:  
        return 0  
    else:  
        return dot_product / (magnitude1 * magnitude2)
```

c. Function that calculates cosine similarity between two word vectors

2. Write a function in Python to compute the Edit Distance between two input strings. For each typo listed above, find the candidate among the top 5 closest to the typo using the Edit Distance function. Assume the cost of insertion, deletion, and substitution to be equal to 1. [4 marks]

```
def editDistance(str1, str2):
    m = len(str1)
    n = len(str2)

    # Create a table to store results of subproblems
    dp = [[0 for x in range(n + 1)] for x in range(m + 1)]

    # Fill d[][] in bottom up manner
    for i in range(m + 1):
        for j in range(n + 1):

            # If first string is empty, only option is to
            # insert all characters of second string
            if i == 0:
                dp[i][j] = j    # Min. operations = j

            # If second string is empty, only option is to
            # remove all characters of second string
            elif j == 0:
                dp[i][j] = i    # Min. operations = i

            # If last characters are same, ignore last char
            # and recur for remaining string
            elif str1[i-1] == str2[j-1]:
                dp[i][j] = dp[i-1][j-1]

            # If last character are different, consider all
            # possibilities and find minimum
            else:
                dp[i][j] = 1 + min(dp[i][j-1],    # Insert
                                   dp[i-1][j],      # Remove
                                   dp[i-1][j-1])    # Replace

    return dp[m][n]
```

a. Edit-Distance with cost 1.

Output: [["boundari", 1], ["transient", 1], ["explain", 2]]

As the cost of insertion, deletion and substitution is same, the cost to convert aerplain to explain and aerplain to aeroplan is both 2 and explain is given as the output, this is fixed using different costs in the next question.

3. Experiment with different costs of insertion, deletion, and substitution (note that all three need not be the same), and identify necessary conditions under which Edit Distance is a valid distance measure. [2 marks]

```
def editDistance(str1, str2):
    m = len(str1)
    n = len(str2)

    # Define costs for insertion, deletion, and substitution
    insert_cost = 3
    delete_cost = 3
    substitute_cost = 4

    # Create a table to store results of subproblems
    dp = [[0 for x in range(n + 1)] for x in range(m + 1)]

    # Fill d[][] in bottom up manner
    for i in range(m + 1):
        for j in range(n + 1):
            # If first string is empty, only option is to
            # insert all characters of second string
            if i == 0:
                dp[i][j] = j * insert_cost # Min. operations = j * insert_cost

            # If second string is empty, only option is to
            # remove all characters of first string
            elif j == 0:
                dp[i][j] = i * delete_cost # Min. operations = i * delete_cost

            # If last characters are same, ignore last char
            # and recur for remaining string
            elif str1[i-1] == str2[j-1]:
                dp[i][j] = dp[i-1][j-1]

            # If last characters are different, consider all
            # possibilities and find minimum
            else:
                dp[i][j] = min(
                    dp[i][j-1] + insert_cost, # Insert
                    dp[i-1][j] + delete_cost, # Remove
                    dp[i-1][j-1] + substitute_cost # Replace
                )

    return dp[m][n]
```

a. Edit-Distance with cost of insertion and deletion = 3 and substitution = 4

Updated Output: [["boundari", 4], ["transient", 4], ["aeroplan", 6]]

Necessary conditions for Edit Distance to be a valid distance measure:

1. Non-negativity: the distance between two strings should always be positive.
2. The distance of a string with itself should always be zero.
3. Symmetry: The distance between string1 and string2 should be the same as distance between string 2 and string 1
4. Triangle inequality should be satisfied

Part 7: WordNet

[Theory + Implementation]

From the NLTK library, use the WordNet interface for the following tasks:

1. Print the list of all synsets corresponding to the words 'progress' and 'advance.'

[1 marks]

Use command “Python wordnet.py” in the project directory to run this code

```
def print_synset_definitions(word):  
    #getting the list of synsets for a word  
    synsets = wordnet.synsets(word)  
    print(f"Synsets for '{word}':")  
    for synset in synsets:  
        print(f" - {synset.name()} : {synset.definition()}")
```

a. Fucntion to print synsets along with definition

Synsets for 'progress':

- advancement.n.03
- progress.n.02
- progress.n.03
- progress.v.01
- advance.v.01
- build\_up.v.02

Synsets for 'advance':

- progress.n.03
- improvement.n.01
- overture.n.03
- progress.n.02
- advance.n.05
- advance.n.06
- advance.v.01
- advance.v.02
- boost.v.04
- promote.v.01
- advance.v.05
- gain.v.05
- progress.v.01
- advance.v.08
- promote.v.02
- advance.v.10
- advance.v.11
- advance.v.12
- advance.s.01
- advance.s.02

2. Print the definitions corresponding to the synsets obtained in the previous question. [1 marks]

Synsets for 'progress':

- advancement.n.03 : gradual improvement or growth or development
- progress.n.02 : the act of moving forward (as toward a goal)

- progress.n.03 : a movement forward
- progress.v.01 : develop in a positive way
- advance.v.01 : move forward, also in the metaphorical sense
- build\_up.v.02 : form or accumulate steadily

Synsets for 'advance':

- progress.n.03 : a movement forward
- improvement.n.01 : a change for the better; progress in development
- overture.n.03 : a tentative suggestion designed to elicit the reactions of others
- progress.n.02 : the act of moving forward (as toward a goal)
- advance.n.05 : an amount paid before it is earned
- advance.n.06 : increase in price or value
- advance.v.01 : move forward, also in the metaphorical sense
- advance.v.02 : bring forward for consideration or acceptance
- boost.v.04 : increase or raise
- promote.v.01 : contribute to the progress or growth of
- advance.v.05 : cause to move forward
- gain.v.05 : obtain advantages, such as points, etc.
- progress.v.01 : develop in a positive way
- advance.v.08 : develop further
- promote.v.02 : give a promotion to or assign to a higher position
- advance.v.10 : pay in advance
- advance.v.11 : move forward
- advance.v.12 : rise in rate or price
- advance.s.01 : being ahead of time or need
- advance.s.02 : situated ahead or going before

3. Estimate the path-based similarity between the words 'advance' and 'progress' using the similarities between their synsets. [2 marks]

```
def path_similarity(word1, word2):
    synsets1 = wordnet.synsets(word1)
    synsets2 = wordnet.synsets(word2)

    max_similarity = 0
    for synset1 in synsets1:
        for synset2 in synsets2:
            #comparing each synset in synset1 to all synsets in synset 2
            similarity = synset1.path_similarity(synset2)
            if similarity is not None and similarity > max_similarity:
                max_similarity = similarity

    return max_similarity
```

- a. Function that returns maximum path based similarity between two synsets

Path-based similarity between 'advance' and 'progress': 1.0

4. Considering that the number of synsets of the words 'advance' and 'progress' are 'm' and 'n,' respectively, what is the number of calls made to the inbuilt path-based similarity function while computing the similarity between the two words? [1 marks]

As each synset in synset1 is being compared to each synset in synset2 the total calls to the path-based similarity function is  $m \times n$ .